M2 — 정적 분석 · 보안 스캔 (제출 산출물)

> 담당 모듈: **M2 (Dev2) — 정적 분석 · 보안 스캔 에이전트** · 포트 `8003` 본 문서는 `backend/m2` 기준으로 0~4번 요구사항을 정리한 제출용 산출물이다.

⚠️ 먼저: M2는 두 형태로 존재한다 (혼동 주의)

	`backend/m2/app/`	`backend/m2/sub2_new/dev2/`
성격	**통합 운영본 (dev 머지 대상 MVP)**	원래 풀 설계 (독립 에이전트)
진입점	`main.py` — `/analyze/static`, `/analyze/security`	`main.py` — `POST /scan` + LangGraph
구성	`main.py` + `analyzer.py` 2개	노드 7개 + RAG + LLM + DB + 코멘트/Slack
코멘트	❌ (m3가 담당)	✅ comment_generate / github_comment
Docker 연결	✅ m1이 호출	❌ 미연결

0번 MVP의 실체는 `app/` 이고, 1·3·4번의 에이전트/LLM/성능평가 설계·구현은 `sub2_new/dev2/` 에 있다. 통합 과정에서 분석 부분만 떼어 `app/`로 경량화했고, 코멘트·판정은 m3로 이관됐다.

---

0. dev 머지 가능한 MVP

머지 대상: `backend/m2/app/` (현재 `feature/module2-static-security` 브랜치에 존재, docker-compose에서 `code-shepherd-m2` 컨테이너로 기동).

동작 보장 범위 (MVP 합격선)

- `POST /analyze/static` — Pylint · Radon(Python) / ESLint(JS·TS) 실행 → 이슈 반환
- `POST /analyze/security` — Bandit · Semgrep(OWASP) 실행 → 이슈 반환
- `GET /health` — 헬스체크
- m1 오케스트레이터가 두 엔드포인트를 호출 → m3 집계까지 E2E 연결됨
- 프론트 "정적 분석·보안 스캔" 페이지에서 결과 실표시

알려진 한계 (오류 허용 범위 — MVP에서 의도적으로 비워둠)

- **Semgrep은 Windows 네이티브 미지원** → graceful skip (`analyzer.py:128 _resolve_semgrep`). 컨테이너(Linux)에서는 동작.
- 도구 미설치/타임아웃(30초) 시 빈 결과로 skip — 파이프라인 중단 없음.
- `context`/`developer` 입력 필드는 받기만 하고 **분석에 미사용** (`main.py:31`은 `pr_diff`만 전달).
- `tools`(도구 실행/스킵 상태)·`stages`(노드별 trace)는 **응답에 미포함** → 프론트는 폴백 표시. (후속 작업 = `docs/tools-stages-integration.md`)
- LLM severity 재판정·RAG·코멘트 생성은 `app/`에 없음(= sub2_new 설계엔 있으나 미통합).

➡️ **결론: 분석→이슈 반환→m1→m3→프론트 표시 E2E가 동작하므로 dev 머지 가능.** 위 한계는 사양 외(out-of-scope) 또는 후속 작업으로 명시.

---

1. 구현 파트 설명 (서비스 / 에이전트 / LLM / 툴 / 워크플로우)

어떤 서비스인가

PR diff를 입력받아 **다국어 정적 분석 + 보안 스캔**을 수행하고, (풀 설계 기준) LLM으로 심각도를 재판단하는 **분석 전담 에이전트 서비스**. MAS 패턴상 **Parallel Independent Agent** — m1이 m3와 함께 병렬로 호출한다.

구성요소 분류 (Agent / LLM / Tool / Workflow)

구분	항목	설명	위치
Tool	Pylint, Radon	Python 정적 분석(품질·순환복잡도)	`app/analyzer.py` / `sub2_new/.../tools/python`
Tool	ESLint	JS·TS 정적 분석	동상 `tools/javascript`
Tool	Bandit	Python 보안(SAST)	`tools/python/bandit_tool.py`
Tool	Semgrep	OWASP Top 10 룰 보안 스캔	`tools/common/semgrep_tool.py`
Tool	pip-audit / npm audit	의존성 CVE 스캔(풀 설계)	`tools/.../npm_audit_tool.py`
LLM Node	llm_judge	severity 재판정 + 오탐 필터(RAG Generate)	`sub2_new/dev2/nodes/llm_judge.py`
LLM Node	comment_generate	skill_level 맞춤 PR 코멘트 생성	`sub2_new/dev2/nodes/comment_generate.py`
Agent	tool_router_agent	PR 맥락 기반 도구 선택	`nodes/tool_router_agent.py`
Agent	action_decider_agent	후속 액션 추천(에스컬레이션 등)	`nodes/action_decider_agent.py`
Workflow	LangGraph StateGraph	7노드 + 조건부 분기 조립	`sub2_new/dev2/graph.py`

LLM 모델 라우팅 (비용 최적화): Critical → `claude-sonnet-4-6`, Major → `claude-haiku-4-5`, Minor → LLM 생략(CLI 판단 신뢰).

에이전트 플로우 (sub2_new 풀 설계 — `graph.py`)

```
START
 → tool_router_agent   (PR 맥락 기반 도구 선택, Agent)
 → run_tools           (선택 도구 병렬 실행, Tool)
 → vector_search       (pgVector RAG Retrieve — 유사 이슈 검색, Tool)
 → llm_judge           (RAG Generate — severity 재판정, LLM)
 → action_decider_agent(후속 액션 추천, Agent)
 → [조건부 분기]
     ├ early_escalation → escalate → store
     └ 그 외             → store
 → store               (PostgreSQL OLTP + BigQuery OLAP 이중 적재)
 → END
```

> "단순 파이프라인"이 아니라 **Agentic Workflow** 인 이유: `route_after_action_decider()`가 LLM/에이전트 판단(`recommended_action`)에 따라 실행 경로를 동적으로 바꾼다(`graph.py:86`).

에이전트 플로우 (app/ 통합 경량본 — 실제 운영)

```
POST /analyze/static|security
 → analyze(pr_diff, agent)            (analyzer.py:293)
    → _parse_diff   : unified diff → post-image 파일 복원
    → _materialize  : 임시 디렉토리에 기록 (지원 확장자만)
    → _run_tools    : 언어별 도구 실행 (Pylint/Radon/ESLint | Bandit/Semgrep)
    → _summarize    : Critical/Major/Minor 집계
 → M2AnalyzeResponse(issues, summary, execution_ms)
```

※ 경량본은 RAG·LLM·에스컬레이션·코멘트 단계 없음(= MVP 범위).

시스템 플로우 (모듈 간)

```
GitHub PR ─webhook→ m1 (오케스트레이터)
   m1 ─POST /analyze/static  ─▶ m2 ─▶ static_result
   m1 ─POST /analyze/security─▶ m2 ─▶ security_result
   m1 ─(m2 2결과 + m3 tests/perf)→ M3ReviewRequest ─▶ m3 POST /review
        m3: 집계 → verdict 판정(lgtm/rework/escalate) → 코멘트 생성 → GitHub 게시 →
DB 저장
   프론트 ◀─ m3 GET /review/{owner}/{repo}/{pr} ─ (m2 이슈 포함)
```

- m1 호출부: `backend/m1/app/clients/module_clients.py:51,75` (httpx, timeout 60s)
- 입력 계약 `M2AnalyzeRequest` / 출력 계약 `M2AnalyzeResponse`: `backend/_shared/shared/contracts.py:66,75`
- 상세 결과 흐름: `backend/m2/docs/result-flow.md` 참고

---

2. 프론트엔드 구현 / 디자인

담당 페이지: **정적 분석 · 보안 스캔** (`frontend/src/pages/security/SecurityPage.tsx`) + 파이프라인 시각화 컴포넌트(`frontend/src/components/M2PipelineFlow.tsx`).

화면 구성 (Notion 테마, 3컬럼)

- **좌측(1/3)**: PR 목록 · 선택 PR 정보(번호/작성자/제출일/판정) · 범례(심각도 색상, 규칙 접두사) · 스캔 도구 상태 카드(데이터 있을 때만)
- **우측(2/3)**: M2 파이프라인 시각화 → 이슈 요약 KPI(Critical/Major/Minor) → 정적/보안 탭 + 이슈 테이블(심각도·위치·규칙·설명·제안)

데이터 연결

- PR 목록: `reviewApi.getTeamPRs(teamId)` (api-core)
- 리뷰 결과: `reviewApi.getReview(owner, repo, prNumber)` → m3 `GET /review` (m2 직접 호출 안 함)
- m2 이슈 구분: `rule_id` 접두사 — `SA-`/`JS-`(정적), `SEC-`/`OWASP-`(보안) (`SecurityPage.tsx:84-85`)
- 심각도 색상 관례: Critical=red, Major=yellow, Minor=blue (`SEVERITY_CLS`)

파이프라인 시각화 (`M2PipelineFlow.tsx`)

- 다크 테마(#0d1117), 4단계 노드: 🔧 도구 실행 → 🔍 벡터 검색 → ⚖️ LLM 판정(Agent) → 💾 결과 저장
- 노드 타입 뱃지: 🤖 `LLM Agent ×1`, ⚙️ `Tool ×3`
- 상태별 색상(success/running/fail/pending) + `pulse-glow`/`pulse-dot` 애니메이션(`src/index.css`)
- `stages?` props로 노드별 status·소요시간 표시 가능 — **현 백엔드 미제공이라 `found`/`isLoading` 기반 폴백** (`stageStatus()`)

---

3. PR 리뷰 코멘트 결과물

현재 운영(app/+m3) 기준

m2는 코멘트를 직접 쓰지 않는다. **m3가 m2 이슈를 포함해 코멘트를 생성·게시**한다 (`backend/m3/app/main.py:341 _review_refine`, `:363 _review_post_comment`). GitHub 토큰 없으면 정직하게 skip(가짜 게시 금지). 프론트는 `review.comment`로 표시.

M2 풀 설계(sub2_new) 코멘트 생성 산출물

`sub2_new/dev2/nodes/comment_generate.py` — enrich ㄴ

출력 예시 (Mock 포맷, `_mock_comment`):

```
## 🔍 코드 리뷰 결과 — PR #42

**개발자 수준**: junior

### 🚨 Critical 이슈
- `src/auth.py:23` — SQL 파라미터 미처리 — f-string으로 쿼리 조합
  - 수정 방법: 파라미터 바인딩(`cursor.execute(sql, params)`) 사용

### ⚠️ Major 이슈
- `src/pay.py:88` — 순환복잡도 18 감지 (함수: process_payment)
```

```
  - 수정 방법: 함수를 더 작은 단위로 분리하세요.

### ℹ️ Minor 이슈
- `src/user.py:10` — 변수명 snake_case 위반: 'userID' → 'user_id'

**요약**: Critical 1건 / Major 1건 / Minor 1건
```

- 코멘트 본문 프롬프트: `sub2_new/dev2/prompts/comment_builder.py`
- GitHub 게시 노드: `sub2_new/dev2/nodes/github_comment.py` (토큰 미설정 시 skip)

---

4. 성능 평가

평가 했는가 — 예 (LLM severity 판정 정확도 평가 구현됨)

- 평가 코드: `sub2_new/dev2/tests/eval/test_severity_eval.py`
- 골든 데이터셋: `sub2_new/dev2/tests/eval/golden_dataset.py` (버전 1.0.0)
- 도구: **LangSmith `evaluate()`** — 케이스별 `llm_judge_node` 호출 → 예측 severity vs 정답 비교 → 0/1 채점

골든 데이터셋 (5케이스: Critical 3 / Major 1 / Minor 1)

#	입력 메시지	CLI 판단	정답	평가 포인트
1	SQL 파라미터 미처리(f-string 조합)	Major	**Critical**	SQL Injection 업그레이드
2	하드코딩 비밀번호 `password='admin123'`	Major	**Critical**	자격증명 노출 업그레이드
3	CVE-2024-1234 (CVSS 9.1)	Major	**Critical**	CVSS 9.x = Critical
4	순환복잡도 18 (process_payment)	Major	**Major**	리팩토링 필요·보안위험 아님(유지)
5	변수명 snake_case 위반(userID)	Minor	**Minor**	코드 스멜(유지)

평가 기준 / 게이팅

- **목표 정확도 ≥ 70%** — 미달 시 `assert` 실패로 머지 차단(`test_severity_eval.py:200`)
- 모델 라우팅 메타: `Critical=sonnet-4-6, Major=haiku-4-5, Minor=skip`
- 결과는 LangSmith 대시보드(`Projects → dev2-{GitHubID}`)에 시계열 기록

실행 조건 (재현 방법)

```
# 실제 LLM 평가 (의미있는 정확도 측정)
export LANGCHAIN_API_KEY=ls__xxxx
export USE_MOCK_LLM=false
export ANTHROPIC_API_KEY=sk-ant-xxxx
pytest dev2/tests/eval/test_severity_eval.py -v -s
```

- `LANGCHAIN_API_KEY` 없으면 LangSmith 평가는 자동 스킵(비용 통제, CI 정상 동작).
- Mock 모드(`USE_MOCK_LLM=true`)는 CLI severity를 그대로 유지 → 업그레이드 케이스(1·2·3)는 0점. 따라서 **실제 정확도는 `USE_MOCK_LLM=false`로 측정해야 의미 있음.**

- Mock 동작 검증 테스트(`test_mock_mode_severity_passthrough`)는 LangSmith 없이 CI에서 항상 실행.

> ⚠️ **주의**: 위 성능평가는 **sub2_new 풀 설계(LLM 포함)** 대상이다. 현재 dev 머지 MVP(`app/`)에는 llm_judge가 없어 severity 재판정 평가는 통합 후(코멘트/LLM 이관 결정 시) 적용 대상이다.

부록: 관련 파일 맵

항목	경로
운영 진입점	`backend/m2/app/main.py`
분석 엔진	`backend/m2/app/analyzer.py`
입출력 계약	`backend/_shared/shared/contracts.py` (M2AnalyzeRequest:66, M2AnalyzeResponse:75)
풀 설계 그래프	`backend/m2/sub2_new/dev2/graph.py`
LLM 판정 노드	`backend/m2/sub2_new/dev2/nodes/llm_judge.py`
코멘트 생성 노드	`backend/m2/sub2_new/dev2/nodes/comment_generate.py`
성능평가	`backend/m2/sub2_new/dev2/tests/eval/{test_severity_eval,golden_dataset}.py`
아키텍처 설계서	`backend/m2/sub2_new/dev2-architecture.md`
결과 흐름 문서	`backend/m2/docs/result-flow.md`
후속 작업 (tools/stages)	`backend/m2/docs/tools-stages-integration.md`
프론트 페이지	`frontend/src/pages/security/SecurityPage.tsx`
프론트 파이프라인	`frontend/src/components/M2PipelineFlow.tsx`